

Spatial Analysis II

One very common issue with spatial analysis is the spatial autocorrelation. Thus, similar values of a variable might be cluster in an specific space, or things that are close together are more alike—or more different—than you'd expect by chance. So, as many spatial analysis assume independence of observation. We need to remove o reduce this autocorrelation.

Usually, we use raster files for many analyses (e.g., Ecological Niche Modelling) and the main problem is that two or more points could be within the same pixel in the raster and extract of have the same value, create redundancy and aslo autocorrelation.

There are different options to reduce spatial autocorrelation, and several specialized packages exist for this purpose. Here, we're going to apply a simple yet effective approach: restricting the number of occurrences using a raster object defined by a specific cell size.

So, let's start calling all of the need libraries and uploading the data.

```
library(raster)
library(terra)
library(sf)

occ <- read.delim("../GBIF/Occ_Final.csv")
head(occ)
```

First, we'll convert the data frame (or matrix) into a spatial point object using the EPSG:4326 (WGS 84) projection. Next, using the extent of our point data, we'll generate a raster with an approximate pixel resolution of 10 km, expressed in decimal degrees (about 10 / 111).

```
occ_sf <- st_as_sf(occ, coords = c("X", "Y"), crs = 4326)

rast_grid <- rast(ext(occ_sf), resolution = (10 / 111), crs = "EPSG:4326")
```

Now, we have a raster with a pixel resolution of approximately 10 km. The next step is to use the *cellFromXY()* function. This function takes the X and Y coordinates of each point and identifies the corresponding pixel (cell) in the raster. As a result, the gridR object will be a vector of cell numbers, each representing the specific pixel that contains the respective coordinate.

Important! Always ensure that the raster object (rast_grid) and the coordinate data (occ_sf) share the same coordinate reference system (CRS).

Finally, we identify and remove the duplicate values, ensuring that each pixel contains only one occurrence.

```
gridR <- cellFromXY(rast_grid, st_coordinates(occ_sf))

unique_cells <- !duplicated(gridR)
occ_thinned <- occ_sf[unique_cells, ]
```

```
length(occ_sf$species)
```

```
## [1] 80773
```

```
length(occ_thinned$species)
```

```
## [1] 7192
```

When we don't need species-specific filtering, this method works well. However, when we require data separated by species, additional steps are necessary. By thinning across pixels regardless of species, we risk removing valuable information—after all, a single pixel might contain data for five different species, yet we would retain only one, losing the other four.

To handle species-specific thinning, we essentially create a loop that repeats the same procedure for each species individually.

```
sp <- unique(occ_sf$species)
out <- matrix(NA, nrow = 1, ncol = length(colnames(occ)))
colnames(out) <- colnames(occ)

for( i in 1:length(sp)){

  occ_tmp <- occ_sf[which(occ_sf$species%in%sp[i]), ]
  gridTmp <- cellFromXY(rast_grid, st_coordinates(occ_tmp))
  occ_tmp <- occ_tmp[!duplicated(gridTmp),]

  out <- rbind( out , cbind(st_drop_geometry(occ_tmp), st_coordinates(occ_tmp)))
}

out <- out[-1, ] #remove empty first row
```

```
length(occ_sf$gbifID)
```

```
## [1] 80773
```

```
length(out$gbifID)
```

```
## [1] 15208
```

Now we can save the final results to a new file.

```
write.table(out, "../GBIF/Occ_Thinned.csv", sep = "\t", row.names = F, quote = F)
```

Two excellent R packages for spatial thinning of point data are GeoThinner and spThin, both widely recognized in ecological modeling.